

Zeev — Command-Line & Syntax Reference

A study guide to the literal syntax this system is built from: shell invocations, .env grammars, the Go/Python wire protocol, ALSA and RTSP device strings, the SQL schema, the HTTP API, and the regex idioms the intent router is written in. Every form below was read out of the source, not the docs. **Placeholders are in <angle brackets>; no real credential appears on this sheet.**

1 • Launch, build and deploy

<code>python3 zeev/zeev.py [--web --https --device] [--no-greeting]</code>	Mode is positional-free — the flag is just searched for in <code>sys.argv</code> . No argparse, so unknown flags are ignored silently.
<code>python3 zeev/zeev.py --call <NUMBER> [--intent "<TEXT>"]</code>	HFP call. <code>--intent</code> seeds what Zeev is trying to accomplish.
<code>./deploy.sh</code>	<code>push</code> → <code>git pull</code> on the Pi → <code>migrate</code> → <code>restart</code> → health-gate → auto-rollback . Requires a clean commit first.
<code>HEALTH_TIMEOUT=120 HEALTH_MARKER='Zeev Device Mode' ./deploy.sh</code>	Both are env overrides. Setting <code>HEALTH_MARKER</code> to a string that never appears is how the rollback path itself gets tested.
<code>cd zeev-audio && make pi # cross-compile only</code> <code>cd zeev-audio && make deploy # build + scp + systemd enable/restart</code> <code>cd zeev-audio && make test # go test ./...</code> <code>G0ARCH=arm64 PI_HOST=ragnar@ragnarok make pi</code>	<code>G00S=linux G0ARCH=arm64</code> — the Pi Zero 2W is Cortex-A53 on 64-bit Raspberry Pi OS, so <code>G0ARM</code> is not needed. Both vars are <code>?</code> = overridable.
<code>ssh ragnar@ragnarok "sudo systemctl stop zeev-audio"</code> <code>scp zeev-audio/bin/zeev-audio-pi</code> <code>ragnar@ragnarok:~/zeev-audio/zeev-audio</code> <code>scp zeev/audio_client.py ragnar@ragnarok:~/Zeev/zeev/audio_client.py</code> <code>ssh ragnar@ragnarok "sudo systemctl start zeev-audio"</code>	The manual daemon deploy. Stop first — the binary is running and scp over a live executable gives ETXTBSY.
<code>ssh ragnar@ragnarok # LAN 10.0.0.168</code> <code>ssh ragnar@ragnarok.tail9c2c7c.ts.net # Tailscale, off-LAN</code>	<code>deploy.sh</code> probes LAN first with <code>-o ConnectTimeout=5 -o BatchMode=yes</code> , then Tailscale.

2 • Reading the machine

<code>journalctl -u zeev-device -f</code>	Follow device mode live.
<code>journalctl -u zeev-audio -f</code>	Follow the Go daemon.
<code>journalctl -u zeev-device --since '-3 min' --no-pager</code>	The exact form <code>deploy.sh</code> greps for its health marker.
<code>journalctl -b -1</code>	Previous boot. Only works because a <code>/etc/systemd/journald.conf.d/</code> file overrides Raspberry Pi OS's <code>Storage=volatile</code> .
<code>sudo journalctl --flush</code>	Journald only starts writing <code>/var/log/journal</code> after a flush — without it the persistence setting looks like it did not take.
<code>systemctl status zeev-device zeev-audio</code>	Read the real state; never assume it.
<code>journalctl -u zeev-device grep 'Zeev \[8B\]'</code>	The raw LLM reply text. Check this before blaming TTS for garbled audio — the 8B romanizes Russian on its own.
<code>journalctl -u zeev-device grep '\[wake\] energy gate'</code>	Frames scored vs frames seen. The tell for a mis-tuned <code>OWW_ENERGY_MULT</code> .
<code>journalctl -u zeev-audio grep 'record: '</code>	<code>record: start max=... silence_rms=...</code> then the finish line with <code>duration / peak_rms / speech</code> . This is how you tell “VAD never fired” from “fired early”.
<code>yt-dlp --version</code>	First check when music silently stops. Must be the <code>/usr/local/bin</code> upstream build, not apt's.
<code>pytest # or: pytest tests/test_intent_gates.py -k hebrew</code>	<code>pytest.ini</code> is at the repo root, so run it from there.

3 · .env — file grammar

Parsed by a hand-written plain-text reader, **not python-dotenv**. KEY=value, one per line; the value runs to end of line and is **not** unquoted, unescaped or interpolated. Never quote a compound value — the quotes become part of it. .env is not in git, so anything here exists only on the machine you typed it on.

Never write .env values with printf. A camera password containing % was eaten by printf and reached the file as 32 spaces plus a fragment. Use an editor. Equally: never put a password inside a URL — @ in a password splits the URL at the wrong place.

3.1 · Compound formats — the ones with real grammar

OWW_MODEL_PATH=<path>.onnx,<path>.onnx	Comma-separated wake-word models. The file stem becomes the prediction key, so renaming a model silently breaks every map keyed on it.
OWW_VOICE_MAP=<stem>:<voice>,<stem>:<voice>	e.g. hey_zeev:daniel,hey_sarina:sarina. Split on , then on the first :: both sides stripped. An entry with no colon is skipped in silence.
OWW_THRESHOLDS=<stem>:<float>,<stem>:<float>	Per-model override of OWW_THRESHOLD. Unlisted stems keep the global. A bad float logs [wake] bad OWW_THRESHOLDS entry and is dropped — it does not raise.
WYZE_CAMERAS=<name>,<name>=rtsp://<host>:554/stream0	Two forms, mixable in one line. A bare name resolves through WYZE_RTSP_BASE; name=<url> is a direct camera. Only the direct-URL ones are swept. Bare names here are also what keeps stream URLs out of prompts sent to Groq/OpenRouter.
ZEEV_SUBJECTS=<name>[<alias>...]:<kind>[:<cam> <cam>]	e.g. smokey smoky smokie:cat:bedroom-cam smokeys-cam. , separates subjects, : separates fields, separates aliases and cameras. kind is what the vision model is asked about; the name never reaches it. Parsed at import, so a typo skips-and-logs rather than stopping the app.
WYZE_RTSP_USER_<NAME>=... WYZE_RTSP_PASS_<NAME>=...	Per-camera override. <NAME> is the stream name with [^A-Za-z0-9]+ → _, stripped and uppercased: bedroom-cam → BEDROOM_CAM. A per-camera user with no per-camera password does not fall back to the shared password — deliberate.
WYZE_RTSP_BASE=rtsp://<user>:<pass>@<host>:8554	The docker-wyze-bridge base. Legacy path; six of eight cameras never answer through it.

3.2 · Full variable table (name = default)

Keys	GROQ_API_KEY TAVILY_API_KEY OPENROUTER_API_KEY GOOGLE_GEOLOC_KEY BOSGAME_KEY CARTESIA_API_KEY OPENAI_API_KEY GEMINI_API_KEY ANTHROPIC_API_KEY	All default "". Groq + Tavily are the only two required for the base app. OPENROUTER_API_KEY gates all vision — it was on the Pi but missing from the dev checkout, so the fallback had never actually run locally.
Providers	LLM_SERVER=groq STT_SERVER=groq TTS_SERVER=auto BOSGAME_URL= BOSGAME_MODEL=llama3.1:8b EMBED_MODEL=nomic-embed-text OLLAMA_URL=http://localhost:11434 OLLAMA_MODEL=llama3.2	BOSGAME_URL is the Cloudflare-proxied origin; the X-Zeev-Key header carries BOSGAME_KEY.
Wake word	OWW_MODEL_PATH= OWW_VOICE_MAP= OWW_THRESHOLDS= OWW_THRESHOLD=0.5 OWW_COOLDOWN=2.0 OWW_SETTLE=1.5 OWW_ENERGY_GATE=1 OWW_ENERGY_MULT=1.8 OWW_ENERGY_MIN=500 OWW_HOLD_FRAMES=25 OWW_PREROLL=20	ENERGY_MULT is 2.5 on the Pi (thermal). PREROLL=20 frames ≈1.6s — it must <i>refill</i> the 16-frame classifier window after reset(), not merely catch the attack.
Recording	ZEEV_RECORD_MAX=20 ZEEV_RECORD_SILENCE_MULT=1.8 ZEEV_VOLUME=90	RECORD_MAX is a ceiling , not a duration — VAD ends the turn early. Raising it is only safe because VAD works.
Cameras	WYZE_CAMERAS= WYZE_RTSP_BASE= WYZE_RTSP_USER= WYZE_RTSP_PASS= WYZE_SNAP_TIMEOUT=25 VISION_TIMEOUT=60 ZEEV_SUBJECTS= ZEEV_SUBJECT_MAX_CAMS=3 ZEEV_SWEEP_MAX_CAMS=3	Both caps exist because one vision call is ~21-25s; an uncapped sweep is minutes.
Behaviour	STREAM_TTS=1 ZEEV_AMBIENT_LOCATION=1 PIPER_MODEL=<path>.onnx	STREAM_TTS=0 reverts to buffered replies; ZEEV_AMBIENT_LOCATION=0 stops sending a coarse city on every turn.
Go daemon	REMOTE_PIPER_URL= REMOTE_PIPER_KEY= REMOTE_PIPER_VOICE= REMOTE_PIPER_URL2= REMOTE_PIPER_KEY2=	Read by the Go process, so they belong in zeev-audio.service's environment. KEY falls back to BOSGAME_KEY when unset; ...2 is the feiergente01 backend that takes odd-numbered sentence chunks.

4 · zeev-audio wire protocol — NDJSON over a Unix socket

Socket `/tmp/zeev-audio.sock`. **One JSON object per line, request and response.** Unknown request fields are ignored, so adding a field is backward-compatible; a missing field is the zero value, which is why every numeric field treats 0 as “use the default”.

<code>{"id": "7", "cmd": "speak_sync", "text": "Hello Alex", "voice": "af_heart"}</code>	→ request
<code>{"id": "7", "ok": true}</code>	← response
<code>{"id": "8", "cmd": "record", "max_seconds": 20, "vad": true, "rate": 16000, "silence_rms": 1800}</code>	→ request
<code>{"id": "8", "ok": true, "wav_b64": "UklGRi..."}</code>	← response

4.1 · Request fields

Field	Type	Meaning
<code>id</code>	string	Echoed back. The client matches responses on it.
<code>cmd</code>	string	One of the 17 below. Unknown → error: unknown command.
<code>text</code>	string	What to speak.
<code>lang</code>	string	en (or empty) → Kokoro/Piper; ru/es → remote Piper only, the local process and backend 2 are English-only.
<code>voice</code>	string	Kokoro voice id — already mapped Python-side (daniel→am_adam, sarina→af_heart). Sending a persona name straight through gets you silent English Piper on one backend and a 500 on the other.
<code>dev</code>	string	ALSA device. Empty → the daemon's current output.
<code>level</code>	int	Volume 0-100 for <code>vol_set</code> .
<code>mac</code>	string	XX:XX:XX:XX:XX:XX for the BT commands.
<code>timeout</code>	int	Seconds, for <code>bt_scan/bt_detect</code> .
<code>max_seconds</code>	float	Recording ceiling . Bounds three things at once (<code>arecord -d</code> , the buffer size, and a $\times 1.5$ wall-clock deadline) — they must move together or the smallest silently wins.
<code>vad</code>	bool	Enable the RMS silence detector, which is the only thing that can end a capture early.
<code>silence_rms</code>	float	int16 RMS below which a frame is silence. Supplied by the caller because the daemon cannot measure it — the mic opens after the user has already started talking. The wake listener publishes the room's rolling median. 0 → built-in default.
<code>rate</code>	int	Sample rate; 16000 for record, the negotiated SCO rate for call audio.
<code>query</code>	string	Search string for play.
<code>block</code>	bool	Wait for playback to finish.

4.2 · Commands and the response fields they set

cmd	Takes	Returns	Notes
speak · speak_sync	text, lang, voice, dev	ok	Kokoro remote → local Piper → espeak-ng. A cancelled speech returns success , not an error — treating it as failure re-spoke the whole reply.
speak_sco	text, rate, dev	ok	Resampled to the SCO rate for a call.
speak_stop	—	ok	Cancels speech. stop only stops music. Sent on its own short-lived socket , or it would queue behind the very speech it is cancelling.
record	max_seconds, vad, rate, silence_rms	ok, wav_b64	arecord + RMS VAD; returns a base64 WAV.
sco_record	rate, dev	ok, wav_b64	Deliberately gets no silence_rms — 8 kHz call audio has a different floor.
vol_get · vol_set	level	ok, level	amixer Master, falling back to the wm8960 Speaker control.
bt_detect · bt_verify	timeout	ok, connected, bt_dev, bt_rate, bt_channels	Parses bluealsa-aplay --list-pcms.
bt_scan	timeout	ok, devices[{mac,name}]	Uses bluez's own --timeout.
bt_pair · bt_connect · bt_disconnect	mac	ok	10s CommandContext bound — an unbounded bluetoothctl leaked a goroutine and a zombie.
play · stop	query, block	ok, title	yt-dlp → ffmpeg → aplay. Python allows 150s and no retry : a retry started a second competing download.
audio_dev	—	ok, dev	The active ALSA PCM string.
health	—	ok	Also what the systemd watchdog self-dials every ~15s.

5 • Shell invocation forms (as the code builds them)

<code>arecord -D <dev> -f S16_LE -r 16000 -c 1 -d <secs></code>	Utterance capture, piped raw. -d is one of the three limits bounding the recording.
<code>arecord -f S16_LE -r 16000 -c 1 -t wav <file></code>	Button-press capture straight to a WAV file.
<code>arecord -D <sco_dev> -f S16_LE -r <rate> -c 1 -q -</code>	Call audio. Mono, at the <i>negotiated</i> SCO rate — usually 8000 or 16000.
<code>aplay -r 22050 -f S16_LE -t raw -</code>	Piper output is raw S16LE at 22050 — no header, so the rate must be stated.
<code>aplay -D <dev> -f S16_LE -r <rate> -c <ch> -q -</code>	Bluetooth. Rate and channels must match the negotiated A2DP format exactly.
<code>aplay -r 44100 -f S16_LE -c 2 -q -</code>	Music: stereo 44.1k.
<code>ffmpeg -loglevel quiet -i pipe:0 -f s16le -ar <rate> -ac 1 pipe:1</code>	WAV in → raw PCM out. Decoding a container.
<code>ffmpeg -loglevel quiet -f s16le -ar <in> -ac 1 -i pipe:0 -f s16le -ar <out> -ac 1 pipe:1</code>	Raw → raw resample. The input format must be declared — raw PCM carries no header, and omitting it makes ffmpeg guess.
<code>ffmpeg -rtsp_transport tcp -i <rtsp_url> -frames:v 1 <out>.jpg</code>	One camera frame. TCP, because UDP loses keyframes.
<code>bluetoothctl --timeout <N> scan on</code>	bluez's own flag, never an outer timeout N wrapper — on bluez 5.82 one-shot scan on exits in ~50 ms, so the wrapper had nothing to kill and every scan returned zero devices.
<code>bluetoothctl devices info <mac> connect <mac> disconnect <mac></code>	All bounded by a 10s context in Go.
<code>bluealsa-aplay --list-pcms</code>	Source of _BT_AUDIO_DEV/_BT_RATE/_BT_CHANNELS. Rate and channels are on the codec line <i>after</i> the device header, and there is a space before Hz.
<code>amixer sset Master <N>% amixer -c wm8960soundcard sset Speaker <raw 0-127> amixer -D bluealsa cset numid=2 50</code>	Three different controls. 90% ≈ raw 114; BT headphones start at raw 50 (~39%).
<code>yt-dlp "ytsearch1:<query>" -o <tmp> -f bestaudio --no-playlist -q --print "before_dl:%(title)s" --extractor-args "youtube:player_client=ios,mweb"</code>	ytsearch1: is the search prefix. The extractor-args line is what works around YouTube's SABR change; --print before_dl gets the title before the download finishes.
<code>piper --model <model>.onnx --output-raw # Go piper --model <model>.onnx --output_raw # Python</code>	Both spellings are live in this codebase and both work — piper accepts the hyphen and the underscore. Worth knowing before you “fix” one of them.
<code>espeak-ng -s 145 -v <lang> --stdout <text></code>	The last-resort voice. -s is words per minute.
<code>pgrep -f 'zeev_music' 'zeev_rec.wav' 'piper --model' 'mpg123'</code>	zeev_cleanup(). -f matches the full command line, which is why the patterns include arguments.
<code>nmcli dev wifi rescan && nmcli dev wifi list</code>	Without the rescan you get NetworkManager's cache — one AP on an idle Pi against a two-AP minimum. Rescan is privileged; the service runs as ragnar and needs a polkit rule.
<code>nmcli device connect bnep0 disconnect bnep0</code>	BT-PAN tethering interface.

6 · Device and URL string syntax

plughw:wm8960soundcard,0	The HAT microphone (_MIC_DEV).
bluealsa:DEV=<MAC>,PROFILE=a2dp	Bluetooth music/speech out. Never wrap it in plug: — the colons in the nested parameters break ALSA's parser. Pass explicit -f/-r/-c to aplay instead.
bluealsa:DEV=<MAC>,PROFILE=sco,SRV=org.bluealsa	Call audio (HFP), BlueALSA v4.3.1+. The SRV= part is required.
rtsp://<user>:<pass>@<host>:554/stream0	Flashed Wyze v3. /stream0 is 1080p ; /stream1 is a 640×360 substream . /live is the v2/Pan path and on v3 it fails as “Invalid data found” with no RTSP dialogue at all — which looks nothing like a wrong path.
rtsp://<user>:<pass>@<host>:8554/<stream>	docker-wyze-bridge. Credentials are WB_API, not WB_PASSWORD — the wrong one gives a bare 401.
rtsp://...:322/...	Secure RTSP does not work with ffmpeg (TLS 1.3 self-signed; fails the handshake with or without verification). Turn it off at the camera.
/tmp/zeev-audio.sock	Daemon socket. Owned by ragnar.
https://<host>/ollama/ · /piper/tts · /piper2/tts	Header X-Zeev-Key: <BOSGAME_KEY>. /piper2/ lives in the ollama.<domain> vhost file, not sites-available/default.

Credentials never go in a logged string. _scrub_rtsp() runs over everything from ffmpeg, because ffmpeg echoes the full input URL — password included — in every error line. It leaked one into a terminal transcript once.

7 · SQLite schema — data/zeev.db (WAL, all writes under _db_lock)

messages(id PK, role, content, ts)	Conversation log. A turn is a user row plus the next assistant row — deleting only one half re-pairs the survivor with an unrelated message.
facts(id PK, fact UNIQUE)	Injected as ## What I know about Alex:. Rewritten inside BEGIN IMMEDIATE + rollback — the old DELETE-then-INSERT lost every fact on a crash between the two.
notes(id PK, text, ts)	From /note and the add_note tool.
settings(key PK, value)	Persisted TTS/volume state.
reminders(id PK, text, due_ts REAL, created_ts REAL, fired INT DEFAULT 0) INDEX idx_reminders_due (fired, due_ts)	due_ts is epoch seconds . Claimed and marked fired in one transaction, because the 20s poll loop can overlap itself and a double announcement is worse than a late one.
message_vecs(message_id PK → messages.id, dim INT, vec BLOB)	vec is little-endian f32. Rows whose dim ≠ the query's are skipped , so changing EMBED_MODEL degrades retrieval instead of corrupting it.
reflections(id PK, period_start, period_end, content, ts)	Weekly reflection.
quantum_insights(id PK, idea, spec_json, result_json, interpretation, ts)	Compounds — the 3 most recent are fed back into the next run.
passages(source, ref, en, he) -- FTS5, in data/torah.db	Chapter granularity only: verses are joined with spaces and no markers, so text cannot be trimmed to a verse. Verse-level rows from import_sefaria.py are the real fix.

8 · HTTP API (--web :5000 / --https :5443, ThreadingHTTPServer)

GET	/memory /notes /volume /battery /health /gps /camera /camera-flip /thermal-status	JSON. /volume is both a GET and a POST.
POST	/chat	SSE stream. Events below.
POST	/clear /memorize /flip /volume /note /delete-note	JSON bodies; /delete-note takes {"idx": N}.
POST	/tts	text → WAV. Returns 503 {"lang":...} when no server-side voice exists, which is the browser's cue to use speechSynthesis.
POST	/transcribe	raw audio → transcript.
POST	/snap	Camera + vision. Sends one token event, not a stream — a half-streamed reply from a model that then 429s cannot be taken back.
POST	/thermal	SSE of MLX90640 frames.

SSE event objects, one JSON per data: line:

{"token": "..."} {"model": "8B"} {"info": "..."} {"error": "..."} {"image": "data:image/jpeg;base64,..."} {"thermal": {"frame": [768 floats °C], "min": ..., "max": ..., "center": ..., "hotspot": ...}}	Reply chunk Which model auto-routing picked Status, e.g. a search in progress Server error Snapshot One thermal frame
---	--

9 · The intent router's own syntax — regex idioms worth reading fluently

Roughly 17 gates decide what an utterance means before any model sees it. They are written in a small set of recurring shapes; learn the shapes and you can read any of them.

^(play put on ...)\s+(?P<query>.+)	Anchored + capture. Must <i>start</i> the utterance, and the named group is the payload. Music, jokes and quantum all use this — that is why “can you maybe play ...” with a lead-in does not match.
\b(scan find)\b.{0,30}\b(bluetooth headphones?)\b	The proximity idiom. .{0,N} between two \b-anchored alternations demands the two ideas appear <i>near</i> each other, which is what stops a keyword from firing across a whole sentence.
m.start() <= 60 # _BT_CALL_MAX_START	Position gates are code, not regex. Call triggers must be in the first 60 chars, goodnight in the first 40, a subject name within 40 of its trigger.
if _T00L_INTENT_RE.search(t): return None	Negative pre-gate. The camera and subject branches sit above the tool branch, so without this “remind me to check on Smokey” would go stare at a camera.
\bcams?\b \bcameras?\b	Always write the plural. \bcam\b and \bcamera\b both fail on “cameras” — that single omission let “check all cameras” fall through to the 8B, which invented two cameras that do not exist.
what(?:'s is) (?:happening going on)	Spell out contractions. Whisper transcribed the same spoken question as “What's going on upstairs?” once and “what is going on upstairs” the next time.
(?m)^\s*[*_] {0,3} \s* (? : Sarina Zeev) \s* : \s*	Strip markup before matching it. The model replied **Sarina:** ; the label pass runs before * is removed, so the pattern has to tolerate the asterisks or the line is spoken as “Sarina colon”.
re.compile(r"[\u0590-\u05FF]") # Hebrew block re.compile(r"[\u0400-\u04FF]") # Cyrillic block	Script ranges, not language detection. The source writes these as literal characters; the escaped form above is the same ranges, and is what you want in any file that might get re-encoded. Note the keyword RAG's <code>_tokenize</code> is <code>\b[a-z]{3,}</code> — ASCII-only, so non-Latin history was never indexed at all. That is why the semantic layer exists.
re.sub(r"^[A-Za-z0-9]+", "_", s).strip("_").upper()	Name → env-var suffix. The one rule to remember when adding a camera.

The lesson these gates keep teaching: an allowlist over unbounded human phrasing **always** has holes, and a miss does not fail safe — it falls through to a model that answers confidently and wrongly. That is why the camera branch also injects a `## Cameras: capability note` into the prompt: the guard has to live on the fallthrough path, not only in the regex.

10 · System configuration syntax (all on the Pi, outside the repo)

systemd	<pre>[Service] Type=notify # zeev-audio; sdnotify sends READY=1 NotifyAccess=main WatchdogSec=30 # self-dials its own socket every ~15s User=ragnar EnvironmentFile=/home/ragnar/Zeev/.env ExecStart=/usr/bin/python3 zeev/zeev.py --device Restart=on-failure RestartSec=5 TimeoutStartSec=60 # Piper ONNX cold load</pre>	zeev-device.service is Type=simple; zeev-audio.service is Type=notify. The watchdog validates the whole accept→dispatch→respond path, so a <i>livelocked</i> daemon gets restarted, not just a crashed one.
cron	<pre>0 6 * * * quantum_daily.py 0 7 * * 0 weekly_reflection.py 0 4 * * 0 /usr/local/sbin/yt-dlp-update.sh # /etc/cron.d/yt-dlp-update</pre>	/etc/cron.d filenames may not contain a dot and the file must be mode 644, or it is ignored silently. yt-dlp -U rewrites the binary, so it must run as root — a user crontab entry fails without sudo.
config.txt	<pre>dtoverlay=i2c-gpio,bus=3,i2c_gpio_sda=5,i2c_gpio_scl=6,i2c_gpi io_delay_us=10</pre>	Software I ² C bus 3 for the MLX90640 at 0x33 on /dev/i2c-3. Hardware /dev/i2c-1 is taken by the WM8960 at 0x19.
polkit	<pre>/etc/polkit-1/rules.d/50-zeev-wifi-scan.rules → org.freedesktop.NetworkManager.wifi.scan</pre>	Scoped to that one action. Without it the Wi-Fi fix silently degrades to the cached scan (and then to a 25 km IP fix) rather than erroring.
journald	<pre>/etc/systemd/journald.conf.d/40-rpi-volatile-storage.conf # same name = override /etc/systemd/journald.conf.d/99-zeev-journal.conf # 200M, 30 days</pre>	Raspberry Pi OS ships Storage=volatile in /usr/lib/...; the override must reuse the same filename in /etc/... to win.
hosts pin	<pre>/usr/local/bin/zeev-lan-hosts.sh + zeev-lan-hosts.timer (60s)</pre>	Pins the Ollama hostname to the LAN IP only while bosgame answers locally , and removes it otherwise — a static hosts entry broke connectivity while travelling.
python	<pre>pip install --break-system-packages <pkg> # PEP 668 pip install webtrcvad-wheels # not 'webtrcvad'</pre>	setuptools 82 dropped pkg_resources, which the original webtrcvad imports. openWakeWord resolves to 0.4.0 on py3.13/aarch64 — newer releases need tflite-runtime, which has no wheel there. Its API is Model(wakeword_model_paths=[...]), not what current docs show.

Study guide · generated 2026-07-31 from zeev/zeev.py, zeev-audio/, deploy.sh, Makefile and the unit files · every value shown is a default or a placeholder — no credential from this system appears here.